Chapter 9. Advanced thread management

This chapter covers

- Thread pools
- Handling dependencies between pool tasks
- Work stealing for pool threads
- Interrupting threads

In earlier chapters, we've been explicitly managing threads by creating `std::thread` objects for every thread. In a couple of places you've seen how this can be undesirable, because you then have to manage the lifetime of the thread objects, determine the number of threads appropriate to the problem and to the current hardware, and so forth. The ideal scenario would be that you could just divide the code into the smallest pieces that can be executed concurrently, pass them over to the compiler and library, and say, "Parallelize this for optimal performance."

Another recurring theme in several of the examples is that you might use several threads to solve a problem but require that they finish early if some condition is met. This might be because the result has already been determined, or because an error has occurred, or even because the user has explicitly requested that the operation be aborted. Whatever the reason, the threads need to be sent a "Please stop" request so that they can give up on the task they were given, tidy up, and finish as soon as possible.

In this chapter, we'll look at mechanisms for managing threads and tasks, starting with the automatic management of the number of threads and the division of tasks between them.

9.1. Thread pools